

ATMs. Let us take an example. If you approach an ATM and do not stand in the position that the system normally recognises, it flags it as an anomaly or unfamiliar sighting.

Smart surveillance is achieved here by giving intelligent sight to the camera itself. If you normally withdraw money and leave, the system does not trigger an anomaly, but if you try to reach out to the ATM in an unfamiliar way, the analytical system evaluates the events based on a scale, which spans from red to green.

Any normal human activity that is expected, for example, standing or walking, would be in the green zone, whereas an unfamiliar or unexpected activity such as you falling down, crouching down or with arms raised up above the head would be in the red zone.

When the system realises that an activity has triggered the red zone, an alert is sent to the responsible personnel or emergency system that it has been programmed to contact. The system can also be taught to detect unauthorised objects that people wear such as helmets or headscarves for early detection of normal behaviour.

Algorithms that learn and adapt

Deep learning may sound fancy but it forms the heart of the system. Uncanny Vision's computer vision has 70-plus algorithms that cover everything from pedestrian detection, vehicle detection, histograms or holography estimation, and is faster than OpenCV (an open source software used for image processing).

Computer vision in Uncanny Vision is targeted to and optimised for an ARM processor. A new set of cameras that Uncanny works on is similar to the ones used in



Fig. 2: Human activity monitoring in green



Fig. 3: Human activity monitoring in red

smartphones as these are equipped with graphic processors and other processing cores for fulfilling all hardware requirements for evaluating the visual sequences captured by these.

Evaluating visual sequences

Processing of data starts after the images reach the CMOS sensor on the camera. From here, these are taken to an image signal processor that cleans up the image from the sensor and then hands it to the main application running on the CPU. All of this happens inside a system-on-chip like Qualcomm's Snapdragon chips used in most smartphones these days.

This is where Uncanny Vision's software takes over and interprets the image using deep learning algorithms and convolutional neural network models. These convolutional neural network software models are able to recognise various useful

information like human actions and faces from video input, and provide the analytics needed for a surveillance system.

The convolutional neural network models are first trained on GPU based Cloud computers using a tool called Caffe with labelled sample data. Uncanny Vision's UncannyDL software uses these models in the camera where the hardware equipped with CPU, GPU and brewed models inside the endpoint device detects any anomalies, and pushes out a photo, video, metadata or a decision to the central Cloud platform.

Caffe is a deep learning framework made with expression, speed and modularity. Uncanny Vision uses Caffe for training convolutional neural network models with maximum accuracy and specificity of whatever it sees and identifies.

Once the images from video footages are in, deep learning Caffe models kick in and refer to 1000 or more objects it has been taught to recognise. The system can also be taught to recognise animals, human poses and defects.

For machines, deep learning has just began

It may sound like science fiction, but in its simplistic form, deep learning is a sub-field of machine learning that works on the learning levels of representations. Representation can be of any form of digital data in form of images, audio, text and the like. For machines to better understand this part, an artificial neural network could be employed to identify physical objects in a multi-level hierarchy of features, factors and concepts.

A convolutional neural network is a kind of deep neural network in the deep learning concept used in this innovation. It has the capability to study and identify various